Computer Vision:

Week-10 (30 March-03 April) Theory Lectures

Dr. Irfan Ali

Assistant Professor (AI, Multimedia Gaming & Cybersecurity

& Program Coordinator of Multimedia & Gaming Deptt

Review

- Machine Learning with vision tasks
- Training, Testing & Validation in ML
- Over/Under fitting and methods to overcome
- Evaluation Performance

Deep Learning Foundations for Vision

With computer vision based applications, Machine Learning (ML) and Deep Learning (DL) represent two distinct approaches to interpreting visual data, differing primarily in how they process features from images. Traditional Machine Learning relies heavily on manual feature engineering, where human experts must pre-define specific characteristics like edges, textures, or shapes (using algorithms like SIFT or HOG) before a classifier can identify an object.

Deep Learning Foundations for Vision

*In contrast, **Deep Learning**—specifically through Convolutional Neural Networks (CNNs)—automates this entire process by using multiple layers to learn hierarchical representations directly from raw pixels, moving from simple lines to complex objects;*

Deep Learning Foundations for Vision

While ML is computationally lighter and highly effective for smaller, structured datasets where interpretability is key, DL scales significantly better with massive amounts of data and typically achieves superior accuracy in complex tasks like facial recognition or autonomous driving,

Deep Learning Foundations for Vision

though it requires substantial GPU power and acts more as a "black box" regarding its internal decision-making logic.

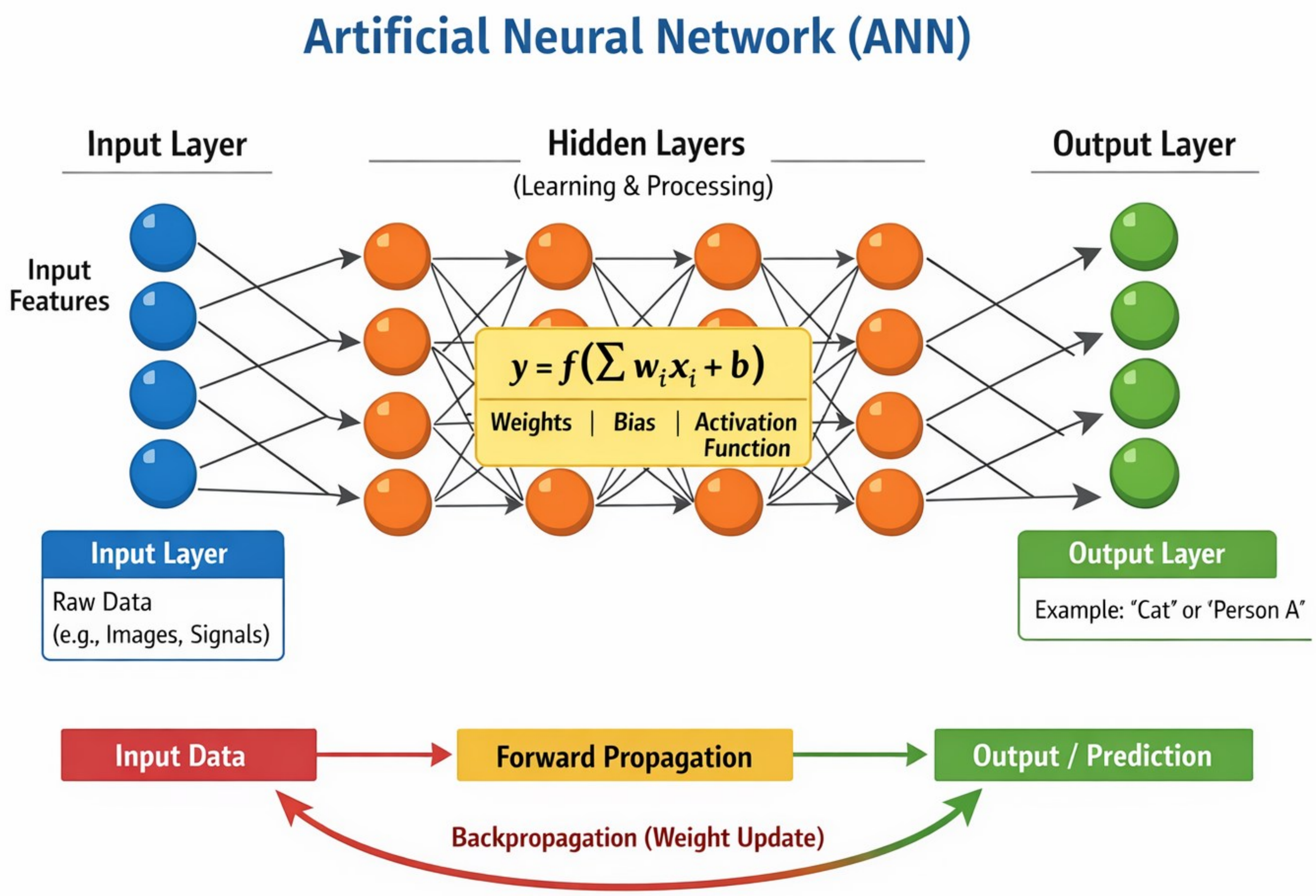
ML & DL

***Machine Learning** is a field of Artificial Intelligence where we give computers the ability to learn from data without being explicitly programmed for every single task.*

***Deep Learning** is a specialized sub-field of Machine Learning inspired by the structure of the human brain, using layers of Artificial Neural Network.*

Artificial Neural Network

*An **Artificial Neural Network (ANN)** is a piece of software designed to mimic the way the human brain processes information. It is the "engine" that powers Deep Learning. Think of it as a large team of digital "neurons" working together to solve a puzzle.*



Artificial Neural Network (ANN)
Input Layer
Hidden Layers
(Learning & Processing)
Output Layer
Input
Features
y = f(∑ w_i x_i + b)
Weights | Bias | Activation Function
Input Layer
Raw Data
(e.g., Images, Signals)
Output Layer
Example: "Cat" or 'Person A"
Input Data
Forward Propagation
Output / Prediction
Backpropagation (Weight Update)

ANN

*How an **ANN** Works-Imagine you are teaching a computer to recognize the number "7" written by hand. The network is organized into three main parts:*

***Input Layer:** The entry point for data. Each neuron in this layer represents a specific feature or attribute of the input signal (e.g., a single pixel value in an image).*

ANN

Hidden Layers: *One or more layers located between the input and output. This is where the "learning" happens; neurons perform weighted sums and apply non-linear activations to extract complex patterns and features from the data.*

Output Layer: *The final layer that produces the network's prediction. The number of neurons here typically corresponds to the number of classes or values the model is designed to identify.*

ANN & CNN

Feature	ANN (Artificial Neural Network)	CNN (Convolutional Neural Network)
Data Input	Requires "Flattened" data (1D vector).	Accepts "Spatial" data (2D or 3D tensors).
Connectivity	Fully Connected: Every neuron in one layer connects to every neuron in the next.	Locally Connected: Neurons only look at a small neighborhood of pixels at a time.
Complexity	High number of parameters; easily overwhelmed by large images.	Fewer parameters due to "weight sharing" (the same filter moves across the image).
Best Use Case	Tabular data (spreadsheets), simple classification, or regression.	Image recognition, video analysis, and medical imaging.

Activation Functions

An **activation function** is a mathematical formula applied to a neuron's output that introduces non-linearity, determining whether and to what extent that neuron should "fire" or be activated based on its input.

Function Name	**Mathematical Form**	**Characteristics & Use Case**
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	Squeezes values between **0 and 1**. Historically popular, now mainly used in the **output layer** for binary classification.

Tanh (Hyperbolic Tangent)	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	Squeezes values between **-1 and 1**. Zero-centered, which often helps the model learn faster than Sigmoid.

ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	Returns 0 for negative inputs and the value itself for positive. The **standard choice** for hidden layers because it is computationally efficient.

Softmax	$$\frac{e^{x_i}}{\sum e^{x_j}}$$	Turns a vector of numbers into a **probability distribution** (sums to 1). Used in the **output layer** for multi-class classification.

Calculate the output of the Sigmoid activation function $\sigma(x)$ when the input $x = 0$.

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\sigma(0) = \frac{1}{1+e^{0}}$$

$$\rightarrow$$

$$\sigma(0) = \frac{1}{1+1} = \frac{1}{2} = 0.5$$

Sigmoid Activation Function

↓

Calculate the output of the Sigmoid activation function sigma(x) when the input x = -1